

Supplementary Information:

CC-SPR: Geometry-Sensitive Sparse Persistent Representatives for Interpretable Omics Feature Selection

Author One^{1,*}, Author Two¹, Author Three¹

¹[Department], [Institution], [City], [Country]

*Corresponding author: author@institution.edu

Contents

S1 Mathematical Background	2
S1.1 Simplicial homology and persistence	2
S1.2 Harmonic representatives and the dense support problem	2
S2 Proofs of Convexity and Sparsity Guarantees	3
S2.1 Convexity of the CC-SPR objective	3
S2.2 Sparsity guarantee	3
S3 Detailed Support Diagnostics Definitions	4
S4 Stability Proposition Under Bounded Metric Perturbation	5
S5 Backend Implementation Details	5
S5.1 Euclidean backend	5
S5.2 Ollivier–Ricci curvature backend	6
S5.3 Diffusion backend	6
S5.4 PHATE-like backend	7
S5.5 Distance-to-measure (DTM) backend	7
S6 Computational Timing Profiles	8
S6.1 Phase 1: Pilot benchmark timing	8
S7 Computational Scaling Analysis	8
S8 Resource-Bounded Single-Cell Validation Details	9
S9 Software Versions	9

S1 Mathematical Background

This section provides the formal mathematical definitions underlying the CC-SPR framework. These are condensed in the main text (Section 2) and presented here in full detail.

S1.1 Simplicial homology and persistence

Let $X = \{x_1, \dots, x_n\} \subset \mathbb{R}^p$ be a point cloud equipped with a metric d . The *Vietoris–Rips complex* at scale ε is

$$\text{Rips}_\varepsilon(X, d) = \{\sigma \subseteq X : \text{diam}_d(\sigma) \leq \varepsilon\}.$$

As ε increases from 0 to ∞ , the nested family $\{\text{Rips}_\varepsilon\}_{\varepsilon \geq 0}$ forms a filtration. Applying simplicial homology with field coefficients $\mathbb{F}$ to this filtration yields a persistence module. By the structure theorem for graded modules over a PID, this module decomposes into a direct sum of interval modules, each corresponding to an interval $[b_i, d_i)$ in the barcode [3].

Definition S1.1 (Persistence barcode). *The barcode of a filtered simplicial complex is the multiset of intervals $\{[b_i, d_i)\}_{i \in I}$ encoding the birth and death scales of homology classes. The persistence (or lifetime) of the i -th feature is $\pi_i = d_i - b_i$, and its midlife is $m_i = (b_i + d_i)/2$.*

Definition S1.2 (Chain groups and boundary maps). *Let K be a simplicial complex. The k -chain group $C_k(K; \mathbb{F})$ is the $\mathbb{F}$ -vector space with basis the k -simplices of K . The boundary operator $\partial_k : C_k \rightarrow C_{k-1}$ is the linear map defined on each k -simplex $\sigma = [v_0, \dots, v_k]$ by*

$$\partial_k(\sigma) = \sum_{i=0}^k (-1)^i [v_0, \dots, \hat{v}_i, \dots, v_k],$$

where $\hat{v}_i$ denotes omission of vertex v_i . The fundamental identity $\partial_{k-1} \circ \partial_k = 0$ ensures that $\text{im}(\partial_{k+1}) \subseteq \ker(\partial_k)$, and the k -th homology group is $H_k(K; \mathbb{F}) = \ker(\partial_k) / \text{im}(\partial_{k+1})$.

S1.2 Harmonic representatives and the dense support problem

For a selected H_1 bar $[b, d)$, let $z_0 \in C_1(\text{Rips}_\varepsilon; \mathbb{R})$ be an initial cycle representative at scale $\varepsilon \in [b, d)$, and let $\partial_2 : C_2 \rightarrow C_1$ denote the boundary operator mapping 2-chains (triangles) to 1-chains (edges). The full set of cycles homologous to z_0 forms the *affine homology class*

$$[z_0] + \text{im}(\partial_2) = \{z_0 + \partial_2 y : y \in C_2\}. \quad (\text{S1})$$

The *harmonic representative* minimizes the ℓ_2 energy within this affine class:

$$\hat{z}_{\text{harm}} = \arg \min_{y \in C_2} \|z_0 + \partial_2 y\|_2^2. \quad (\text{S2})$$

This is a standard least-squares problem whose solution is the orthogonal projection of z_0 onto $\ker(\partial_1) \cap (\text{im}(\partial_2))^\perp$, coinciding with the harmonic representative in the Hodge-theoretic sense when

the chain complex is equipped with the standard inner product [1, 2].

Remark S1.3 (Dense support problem). *The solution to (S2) typically has $\|\hat{z}_{\text{harm}}\|_0 = O(|E|)$ where $|E|$ is the number of edges at scale ε . In omics applications where n ranges from 10^2 to 10^4 , this can mean hundreds or thousands of nonzero edge weights, diluting the biological signal when projected to feature space. The harmonic representative finds the smoothest cycle, not the most parsimonious one.*

S2 Proofs of Convexity and Sparsity Guarantees

This section provides the formal proofs referenced in the main text (Section 2.2).

S2.1 Convexity of the CC-SPR objective

Proposition S2.1 (Convexity). *The CC-SPR objective*

$$f(y) = \|z_0 + \partial_2 y\|_2^2 + \lambda_1 \|z_0 + \partial_2 y\|_1 + \lambda_2 \|y\|_2^2 \quad (\text{S3})$$

is convex in y . When $\lambda_2 > 0$, it is strongly convex with a unique minimizer.

Proof. Each term is a convex function of y :

1. $f_1(y) = \|z_0 + \partial_2 y\|_2^2$ is convex as a composition of a convex quadratic with the affine map $y \mapsto z_0 + \partial_2 y$;
2. $f_2(y) = \lambda_1 \|z_0 + \partial_2 y\|_1$ is convex because the ℓ_1 norm is convex and composition with affine maps preserves convexity;
3. $f_3(y) = \lambda_2 \|y\|_2^2$ is convex (strongly convex when $\lambda_2 > 0$).

A nonnegative sum of convex functions is convex, and strong convexity of any summand implies strong convexity of the sum. $\square$

Remark S2.2 (Connection to elastic net regression). *The structure of (S3) mirrors the elastic net [4]: λ_1 plays the role of the LASSO penalty promoting sparsity in the edge weights, while λ_2 provides the ridge-like stability that prevents ill-conditioning when ∂_2 is rank-deficient. The key difference is that the “design matrix” here is the boundary operator ∂_2 rather than a data matrix, and the “response” is the initial cycle z_0 .*

S2.2 Sparsity guarantee

Proposition S2.3 (Sparsity guarantee). *For any $\lambda_1 > 0$, the solution $\hat{z}_{\text{spr}}$ satisfies*

$$\|\hat{z}_{\text{spr}}\|_0 \leq \|\hat{z}_{\text{harm}}\|_0,$$

with equality only when the harmonic representative already has minimal ℓ_1 norm in its homology class.

Proof. The ℓ_1 penalty acts as a soft-thresholding operator on the edge weights of the representative. Any edge weight $|z_e| < \tau(\lambda_1)$ for an appropriate threshold τ is driven to zero. Since the harmonic representative has no such thresholding, its support is at least as large. Equality requires that every nonzero weight in $\hat{z}_{\text{harm}}$ exceeds $\tau(\lambda_1)$, which occurs only when the harmonic solution already has minimal ℓ_1 norm—a non-generic condition. $\square$

S3 Detailed Support Diagnostics Definitions

Beyond TIP (defined in the main text, Section 2.3), the quality of a sparse representative can be characterized by several support diagnostics used throughout the results.

Definition S3.1 (Support size). *For a representative $\hat{z} \in \mathbb{R}^{|E|}$, the support size is $\|\hat{z}\|_0 = |\{e \in E : \hat{z}_e \neq 0\}|$. After projection to feature space, the projected support size is $|\{j \in \{1, \dots, p\} : |w_j| > 0\}|$ where $w \in \mathbb{R}^p$ is the projected feature-importance vector.*

Definition S3.2 (Representative entropy). *Let $\hat{p}_e = |\hat{z}_e| / \sum_{e'} |\hat{z}_{e'}|$ be the normalized absolute edge weights. The representative entropy is*

$$H(\hat{z}) = - \sum_{e: \hat{z}_e \neq 0} \hat{p}_e \log \hat{p}_e.$$

Low entropy indicates concentrated support (weight localized on a few edges); high entropy indicates diffuse support (weight spread across many edges).

Definition S3.3 (Gini coefficient of representative weights). *The Gini coefficient of the weight distribution provides a complementary inequality measure:*

$$G(\hat{z}) = \frac{\sum_{i=1}^m \sum_{j=1}^m ||\hat{z}_{e_i}| - |\hat{z}_{e_j}||}{2m \sum_{i=1}^m |\hat{z}_{e_i}|},$$

where $m = \|\hat{z}\|_0$ is the support size. High Gini indicates that weight is concentrated on a few edges; low Gini indicates uniform distribution.

Remark S3.4 (Interpreting TIP entropy and Gini in results tables). *In the main text Tables 3–4 and Table 7, the “TIP Ent.” and “TIP Gini” columns report the entropy and Gini coefficient of the TIP profile itself (not of the raw edge weights). In omics, a representative with high Gini coefficient and low entropy concentrates its topological signal on a small number of sample-pair relationships, which project to a compact gene set. This concentration is what makes the representative biologically actionable: it identifies a gene module rather than a diffuse background signal. The ℓ_1 penalty in the CC-SPR objective directly promotes this concentration by driving small weights to zero.*

S4 Stability Proposition Under Bounded Metric Perturbation

This section formalizes the theoretical guarantee that backend choice can be studied as a controlled perturbation of topological output, referenced in the main text (Section 2.4).

Proposition S4.1 (Filtration stability under backend perturbation). *Let $D^{(a)}$ and $D^{(b)}$ be distance matrices from two backends. Define $\varepsilon_\infty = \|D^{(a)} - D^{(b)}\|_\infty$. Then the Rips filtrations satisfy an ε_∞ -interleaving:*

$$\text{Rips}_r(X, D^{(a)}) \subseteq \text{Rips}_{r+\varepsilon_\infty}(X, D^{(b)}) \subseteq \text{Rips}_{r+2\varepsilon_\infty}(X, D^{(a)})$$

for all $r \geq 0$. Consequently, the bottleneck distance between the corresponding persistence diagrams satisfies

$$d_B(\text{Dgm}(D^{(a)}), \text{Dgm}(D^{(b)})) \leq \varepsilon_\infty.$$

Proof. The first inclusion follows because if $\text{diam}_{D^{(a)}}(\sigma) \leq r$, then $\text{diam}_{D^{(b)}}(\sigma) \leq r + \varepsilon_\infty$ by the definition of $\varepsilon_\infty = \max_{i,j} |D_{ij}^{(a)} - D_{ij}^{(b)}|$. The second inclusion follows symmetrically. The bottleneck bound is the standard algebraic stability theorem for Rips filtrations applied to ε_∞ -interleaved persistence modules [5]. $\square$

Remark S4.2 (Practical implication). *This proposition means that backend choice can be studied as a controlled perturbation of the topological output. When ε_∞ is small relative to persistence values, the barcode is approximately preserved; when it is large, the backend is genuinely changing the topological signal. The benchmark results in the main text can be understood through this lens: backends that produce substantially different distance matrices yield different topological features and different downstream classifier performance.*

Remark S4.3 (Bridge edges, bottleneck sensitivity, and backend sharpening). *Backends that preferentially upweight bottleneck or bridge-like edges—as Ricci flow does for negatively curved edges—can sharpen the persistent signal by increasing the filtration parameter at which spurious shortcuts create topological noise. This is the mechanism by which curvature-aware backends can improve representative quality: by delaying the merger of distinct topological components, they preserve genuine cycles at the expense of noise-induced ones. The effect is data-dependent and non-monotone, as demonstrated by the divergent LUAD/GSE161711 results across the geometry family.*

S5 Backend Implementation Details

This section provides the full implementation specifications for each geometry backend, summarized in the main text (Section 2.4).

S5.1 Euclidean backend

Standard ℓ_2 distance in PCA space: $D_{ij}^{(\text{eu})} = \|x_i - x_j\|_2$. Applied after optional PCA dimensionality reduction. Advantages: simplicity, interpretability, computational efficiency. Limitation: treats all

directions equally and does not adapt to local density or curvature.

S5.2 Ollivier–Ricci curvature backend

Definition S5.1 (Ollivier–Ricci curvature on graphs [6, 7]). *Let $G = (V, E, w)$ be a weighted graph. For each vertex v , define a probability measure μ_v supported on its neighbors (e.g., uniform on the k -nearest neighbors). The Ollivier–Ricci curvature of an edge $(u, v) \in E$ is*

$$\kappa(u, v) = 1 - \frac{W_1(\mu_u, \mu_v)}{d(u, v)},$$

where W_1 denotes the Wasserstein-1 (earth mover’s) distance.

Implementation procedure:

1. Construct k -nearest-neighbor graph $G = (V, E, w)$ with $w_{ij} = \|x_i - x_j\|_2$.
2. For T iterations: compute Ollivier–Ricci curvature $\kappa(i, j)$ for each edge via optimal transport between neighbor distributions; update weights $w_{ij}^{(t+1)} = w_{ij}^{(t)} - \eta \cdot \kappa(i, j) \cdot w_{ij}^{(t)}$.
3. Compute shortest-path distances on the deformed graph.

Default parameters: $k = 10$, $T = 5$, $\eta = 0.5$.

Positive curvature indicates convergent geometry (neighborhoods of u and v are close); negative curvature indicates divergent geometry characteristic of bridges, bottlenecks, or saddle-like structure between distinct communities. Ricci flow upweights negatively curved (bridge) edges, delaying merger of distinct components in the filtration, preserving genuine cycles at the expense of noise-induced shortcuts.

Remark S5.2 (Non-monotonicity of Ricci benefits). *Ricci flow does not uniformly improve downstream performance. It helps when curvature correction suppresses misleading shortcut edges (as on LUAD, where Ricci achieves $F1 = 0.5012$ vs Euclidean = 0.4342). It does not help when Euclidean neighborhoods already capture the relevant geometry (as on GSE161711, where Euclidean achieves $F1 = 0.7483$ vs Ricci = 0.5944). This non-monotonicity motivates treating geometry as a diagnostic axis.*

S5.3 Diffusion backend

Definition S5.3 (Diffusion distance [9]). *Let W be a weight matrix derived from a Gaussian kernel $W_{ij} = \exp(-\|x_i - x_j\|^2/\sigma^2)$, and let $P = D_W^{-1}W$ be the row-normalized Markov transition matrix where $D_W = \text{diag}(\sum_j W_{ij})$. The diffusion distance at time t is*

$$D_t^{(\text{diff})}(i, j) = \left(\sum_{\ell \geq 1} e^{-2t\mu_\ell} (\psi_\ell(i) - \psi_\ell(j))^2 \right)^{1/2},$$

where μ_ℓ and ψ_ℓ are eigenvalues and right eigenvectors of P .

The diffusion time parameter t controls the scale of geometry: small t recovers local Euclidean-like distances; large t emphasizes global connectivity. Diffusion distances can equivalently be expressed through the heat kernel $H_t = e^{-tL}$ as $D_t^{(\text{diff})}(i, j)^2 = H_t(i, i) + H_t(j, j) - 2H_t(i, j)$, connecting persistent homology to graph spectral methods.

Implementation: truncated eigendecomposition via `scipy.sparse.linalg.eigsh` with 50 eigenvectors.

S5.4 PHATE-like backend

Definition S5.4 (PHATE-like potential distance [10]). *Let P^t be the t -step transition matrix. The potential representation of sample i is $U_i = -\log(P_{i,\cdot}^t + \epsilon)$, where $\epsilon > 0$ is a regularization constant. The potential distance is*

$$D^{(\text{phate})}(i, j) = \|U_i - U_j\|_2.$$

The log transformation converts transition probabilities into surprisal (information content). The potential distance therefore measures how differently the random walks emanating from i and j explore the graph, related to the symmetrized KL divergence between diffusion distributions. In information-geometric terms, $D^{(\text{phate})}$ approximates a Fisher–Rao-like distance on the statistical manifold of diffusion distributions.

Implementation: lightweight potential-distance computation without the full MDS step; $\epsilon = 10^{-7}$ for numerical stability. Results may differ from those obtained with the complete PHATE pipeline including MDS dimensionality reduction.

S5.5 Distance-to-measure (DTM) backend

Definition S5.5 (Distance to measure [11, 12]). *For a point cloud X with empirical measure $\mu_X = \frac{1}{n} \sum_{i=1}^n \delta_{x_i}$ and a mass parameter $m_0 \in (0, 1]$, the distance-to-measure function at point x is*

$$d_{\mu_X, m_0}(x) = \left(\frac{1}{\lceil m_0 n \rceil} \sum_{j=1}^{\lceil m_0 n \rceil} \|x - x_{(j)}\|^2 \right)^{1/2},$$

where $x_{(1)}, \dots, x_{(n)}$ are the points of X sorted by distance to x .

Proposition S5.6 (Stability of DTM [11]). *If μ and ν are two probability measures on $\mathbb{R}^p$, then*

$$\|d_{\mu, m_0} - d_{\nu, m_0}\|_\infty \leq \frac{1}{\sqrt{m_0}} W_2(\mu, \nu),$$

where W_2 is the Wasserstein-2 distance.

DTM-weighted distances delay inclusion of outlier-adjacent simplices, suppressing topological noise from density spikes. Implementation: k NN-based measure estimation for computational efficiency.

S6 Computational Timing Profiles

S6.1 Phase 1: Pilot benchmark timing

Table S1 reports per-backend runtimes for the Phase 1 pilot benchmark (LUAD pilot $n = 60$, GSE161711 $n = 96$, GSE165087 bounded $n = 320$) on an HPC cluster (48 GB, 8 CPUs).

Dataset	Backend	Seconds
LUAD ($n = 60$)	euclidean	0.5
LUAD	ricci	5.2
LUAD	diffusion	0.5
LUAD	phate_like	0.2
LUAD	dtm	0.2
GSE161711 ($n = 96$)	euclidean	8.0
GSE161711	ricci	29.0
GSE161711	diffusion	2.5
GSE161711	phate_like	1.2
GSE161711	dtm	14.4
GSE165087 ($n = 320$)	euclidean	1099.0
GSE165087	ricci	6.2
GSE165087	diffusion	1.9
GSE165087	phate_like	36.3
GSE165087	dtm	3932.1

Table S1: Per-backend runtime in seconds for the Phase 1 pilot benchmark (HPC; 48 GB, 8 CPUs). These three datasets completed in approximately 3.3 hours total. The DTM backend on GSE165087 is the most expensive computation at ~ 3932 seconds due to k NN-based measure estimation on $n = 320$ cells.

S7 Computational Scaling Analysis

The full-scale LUAD evaluation was conducted on a local workstation (Intel i9-9920X, 24 threads @ 3.50 GHz, 125 GB RAM). Table S2 reports per-backend walltimes, revealing dramatic nonlinear scaling from pilot to full scale.

Backend	Pilot ($n = 60$)	Full ($n = 275$)	Scaling	Bottleneck
euclidean	0.5 s	2,527 s (42 m)	$\sim 5054\times$	CVXPY cycle opt.
ricci	5.2 s	8,896 s (2.5 h)	$\sim 1711\times$	Curvature flow + CVXPY
diffusion	0.5 s	7,851 s (2.2 h)	$\sim 15701\times$	Spectral decomp. + CVXPY
phate_like	0.2 s	2,100 s (35 m)	$\sim 10500\times$	Log-potential + CVXPY
dtm	0.2 s	6,487 s (1.8 h)	$\sim 32435\times$	k NN measure + CVXPY

Table S2: Walltime scaling from pilot ($n = 60$, 2-split, HPC 8 CPUs) to full scale ($n = 275$, 5-fold, local i9-9920X 24 threads). The superlinear scaling reflects three compounding factors: (i) the distance matrix grows as $O(n^2)$; (ii) the Rips complex grows combinatorially, increasing both persistence computation and boundary-matrix size for CVXPY; (iii) 5-fold CV vs 2-split increases solver invocations by $\sim 2.5\times$. TIP gene extraction adds an additional 15–150 minutes per backend (not included).

The extreme scaling factors (up to $32,000\times$ for DTM) underscore that CC-SPR’s cycle optimization is the computational bottleneck, not the geometry backend itself. The CVXPY elastic-net solver operates on boundary matrices whose size grows with the number of edges in the Rips complex, which scales superlinearly with n . Future work on scalability should target approximate cycle representatives or subsampled boundary matrices rather than backend optimization alone.

Practical recommendation. For exploratory analyses, the pilot protocol ($n \leq 100$, 2–3 splits) completes in seconds and suffices for backend selection. For definitive results, the full-scale protocol ($n = 275$, 5-fold CV) requires 2–3 hours per backend on a modern workstation and should be parallelized across backends.

S8 Resource-Bounded Single-Cell Validation Details

Three larger GSE165087 configurations failed under cluster memory limits, establishing the resource boundary for single-cell analysis. Table S3 reports the full execution envelope.

Run type	State	Elapsed	Requested memory	Peak RSS
scRNA full	OUT_OF_MEMORY	16:43:25	96 GB	100.3 GB
scRNA fast-track	TIMEOUT	20:00:06	96 GB	75.2 GB
scRNA low-memory	OUT_OF_MEMORY	11:25:59	48 GB	49.7 GB
scRNA manuscript-safe	COMPLETED	00:07:49	32 GB	2.9 GB
scRNA ablation-safe	COMPLETED	—	32 GB	~9.3 GB

Table S3: Execution envelope for single-cell experiments on GSE165087. The manuscript-safe configuration ($n = 320$ cells) produces $F1 = 0.925$ for all methods (ceiling effect). The ablation-safe configuration ($n = 400$) shows CC-SPR (0.823) exceeding harmonic (0.798) while trailing standard (0.888).

S9 High-Bootstrap Stability and Arabidopsis Feasibility

This section documents the higher- n_{boot} stability experiments reported in the main text and explains the computational limits that prevented matching n_{boot} across datasets.

S9.1 GSE161711 at $n_{\text{boot}} = 1000$

All five backends were run on 5,000 variable genes with $n_{\text{boot}} = 1000$ and 0.8 subsample fraction. Completion rates, concentration metrics, and per-backend wall times are reproduced in the main text (Table ?? of the main manuscript). The label-permutation test used the Euclidean backend (the cheapest for this dataset), $n_{\text{perm}} = 200$, $n_{\text{splits}} = 5$, and $n_{\text{boot}} = 5$ per permutation. Observed $F1 = 0.6876$; zero of 200 permutations matched or exceeded it, yielding an empirical p -value of 0.0000 (95% upper CI ≈ 0.015 under the rule-of-three approximation). Total wall time across the five backends plus permutation test was approximately 8.3 hours on a 12-core/24-thread Intel i9-9920X with 125 GB RAM (CPU only).

S9.2 Arabidopsis scalability limits

The original Arabidopsis benchmark in the main text uses $n_{\text{boot}} = 3$ within 3-fold CV on 500 cells. For the stability study we attempted to scale to $n_{\text{boot}} = 1000$ following the GSE161711 protocol. Three successive attempts revealed a hard scalability frontier.

- **Attempt 1** ($n = 2,999$ cells, $n_{\text{boot}} = 20$): OOM-killed at ~ 71 GB peak RAM on the 125 GB workstation during the first timing-probe iteration. Memory was still climbing at the kill point. Root cause: the Rips complex on a 2,399-cell subsample in 50 PCA dimensions stores $O(n^3) \approx 1.4 \times 10^{10}$ potential triangles, dominating memory.
- **Attempt 2** ($n = 1,000$ cells, $n_{\text{boot}} = 20$): timing probe completed with stable memory at ~ 14 GB, but per-iteration cost reached 12,084 s (3.4 h). Extrapolating, $n_{\text{boot}} = 20$ would take ~ 67 hours. The CVXPY solver emitted “Solution may be inaccurate” warnings, indicating ill-conditioned boundary matrices on the dense PCA distance matrix. Killed after timing probe.
- **Attempt 3** ($n = 500$ cells, $n_{\text{boot}} = 50$): initial parallel runs on Euclidean+DTM and Ricci+Diffusion+PHATE-like hung without producing checkpoints after more than 20 hours of accumulated CPU time; timing probes had suggested 31–204 s/iter, but actual per-iteration cost on pathological subsamples was > 1000 s. Both runs were killed.
- **Attempt 3b** ($n = 500$ cells, $n_{\text{boot}} = 10$, **Euclidean + DTM**): *succeeded*. Euclidean: 10/10 iterations, support = 46, entropy = 3.655, Gini = 0.323, elapsed 10,189 s (2.83 h). DTM: 10/10, support = 48, entropy = 3.733, Gini = 0.289, elapsed 11,609 s (3.22 h). Total wall time ≈ 6 h.

Why Arabidopsis is pathologically slower than GSE161711. GSE161711 is a bulk dataset with $n = 96$ samples in a sparse gene-space distance matrix. Arabidopsis is represented in 50 PCA dimensions, producing a *dense* distance matrix where every cell pair is non-trivially close. This has three consequences: (i) many edges and triangles enter the Rips filtration at low scales, inflating the boundary matrix passed to CVXPY; (ii) the elastic-net cycle problem is ill-conditioned on dense boundary matrices, triggering many solver iterations; (iii) per-iteration cost varies dramatically with the random subsample geometry, so probe timings can under-estimate the realized run time by a factor of 20–30 $\times$. The backend slowdown versus GSE161711 ranges from $\sim 9\times$ (DTM) to $\sim 227\times$ (Diffusion).

Why a reduced cell count is still a valid stability test. Reducing from 2,999 $\rightarrow$ 500 cells yields a *conservative* stability assessment: fewer cells produce noisier geometry estimates, so TIP scores have higher bootstrap-to-bootstrap variance, making it harder (not easier) to demonstrate stability. Any stability observed at 500 cells is therefore a lower bound on the method’s stability

at full scale. The reduction does not affect the validity of the classification F1 results reported in the main text, which use the original $n = 2,999$ -cell benchmark from `run_arabidopsis.py`.

Methodological implication. The Arabidopsis scalability frontier is fundamental to the exact-Rips + elastic-net approach, not an implementation bug. Scaling CC-SPR to large single-cell atlases will require either approximate persistent homology (sparse Rips, witness complexes) or better-conditioned cycle optimizers (for example, L1 regression or first-order methods), both of which are identified as future work.

S10 Software Versions

Package	Version
Python	≥ 3.10
GUDHI	≥ 3.8
CVXPY	≥ 1.4
GraphRicciCurvature	≥ 0.5
scanpy	≥ 1.9
scikit-learn	≥ 1.3
gseapy	≥ 1.1
numpy	≥ 1.24

Table S4: Key software dependencies. A frozen environment file (`requirements-frozen.txt`) with exact versions is provided in the repository.

References

- [1] D. Gurnari, A. Guzman-Saenz, F. Utro, A. Bose, S. Basu, and L. Parida. Probing omics data via harmonic persistent homology. *Scientific Reports*, 15:3422, 2025.
- [2] J. Friedman. Computing Betti numbers via combinatorial Laplacians. *Algorithmica*, 21(4):331–346, 1998.
- [3] A. Zomorodian and G. Carlsson. Computing persistent homology. *Discrete & Computational Geometry*, 33(2):249–274, 2005.
- [4] H. Zou and T. Hastie. Regularization and variable selection via the elastic net. *JRSS Series B*, 67(2):301–320, 2005.
- [5] F. Chazal, V. de Silva, M. Glisse, and S. Oudot. The structure and stability of persistence modules. *Springer*, 2016.
- [6] Y. Ollivier. Ricci curvature of metric spaces. *Comptes Rendus Mathematique*, 345(11):643–646, 2007.

- [7] Y. Ollivier. Ricci curvature of Markov chains on metric spaces. *Journal of Functional Analysis*, 256(3):810–864, 2009.
- [8] C.-C. Ni, Y.-Y. Lin, F. Luo, and J. Gao. Community detection on networks with Ricci flow. *Scientific Reports*, 9:9984, 2019.
- [9] R. R. Coifman and S. Lafon. Diffusion maps. *Applied and Computational Harmonic Analysis*, 21(1):5–30, 2006.
- [10] K. R. Moon, D. van Dijk, Z. Wang, et al. Visualizing structure and transitions in high-dimensional biological data. *Nature Biotechnology*, 37(12):1482–1492, 2019.
- [11] F. Chazal, D. Cohen-Steiner, and Q. Mérigot. Geometric inference for probability measures. *Foundations of Computational Mathematics*, 11(6):733–751, 2011.
- [12] F. Chazal, B. Fasy, F. Lecci, B. Michel, A. Rinaldo, and L. Wasserman. Robust topological inference: distance to a measure and kernel distance. *JMLR*, 18(159):1–40, 2017.
- [13] L. Haghverdi, M. Büttner, F. A. Wolf, F. Buettner, and F. J. Theis. Diffusion pseudotime robustly reconstructs lineage branching. *Nature Methods*, 13(10):845–848, 2016.